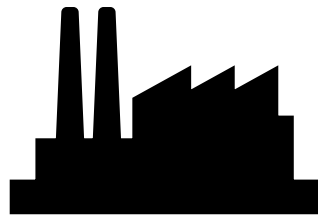


R5.07

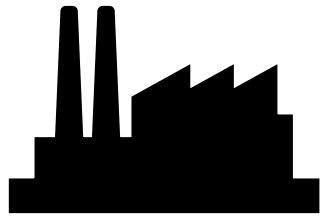
Automatisation de la chaîne de production

Jean-Yves Colin
IUT du Havre
Département Informatique
v1.03- 2023-2025

L'« industrialisation » de l'informatique

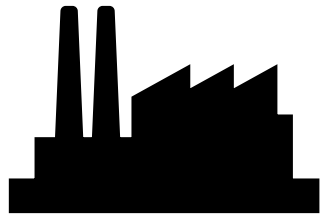


L'« industrialisation » de l'informatique



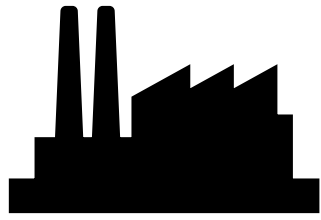
- Buts :
 - Rapidité de développement (« vite »)
 - Qualité du travail rendu (« bien »)
- Outils pour y parvenir :
 - Bibliothèques riches et nombreuses
 - Travail en équipe
 - Méthodologies de développement
 - Nombreux contrôles

L'« industrialisation » de l'informatique



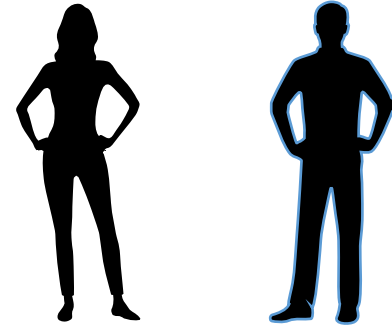
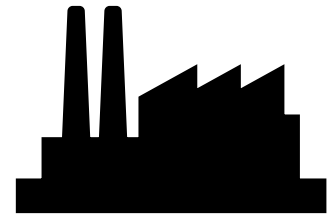
- Bibliothèques riches et nombreuses
 - Elles fournissent des codes bas ou moyen niveaux déjà écrits et testés
 - développement plus rapide
 - moins de risques d'erreurs
 - Parfois, l'existence d'une bibliothèque appropriée XYZ suffit pour décider d'utiliser un langage (« il possède la bibliothèque XYZ ») plutôt qu'un autre langage (« oui, il est mieux, mais il ne possède pas la bibliothèque XYZ ou une équivalente »)

L'« industrialisation » de l'informatique



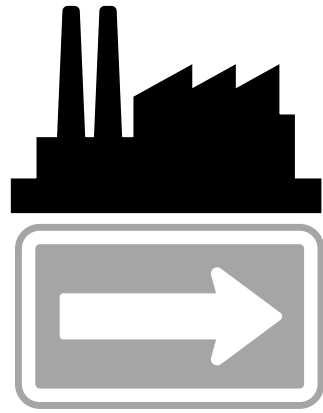
- Bibliothèques riches et nombreuses (suite)
 - Par conséquent, pour un développeur, la maîtrise de certaines bibliothèques est devenue peu à peu aussi importante que la connaissance d'un langage. Il lui faut alors maîtriser :
 - La logique de la construction de la bibliothèque (structure, passage des paramètres, retour des résultats, etc.)
 - Les effets des choix internes sur les programmes qui l'utilisent (performances, gestion des erreurs, etc.)
- Synonyme : 'framework'

L'« industrialisation » de l'informatique



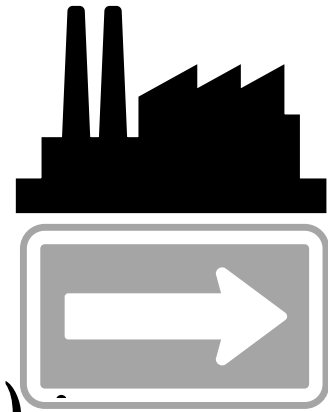
- Plus travail en équipe
 - Ce qui implique, entre autres :
 - découpage du travail
 - réunions de discussion
 - répartition des responsabilités
 - outils de communications
 - outils de partage
 - etc.

L'« industrialisation » de l'informatique



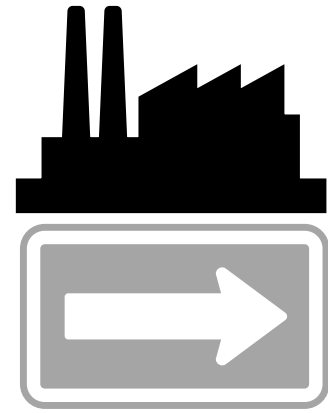
- plus utilisation de méthodologies de développement
 - langages
 - « fortement typés » plutôt que « faiblement typés »
 - « à Objets »
 - etc.
 - ...

L'« industrialisation » de l'informatique



- plus utilisation de méthodologies de développement (suite) :
 - ...
 - « *workflow* » : un **flux de travaux** ou encore **flux opérationnel**, est la représentation d'une **suite de tâches** ou d'**opérations** effectuées par une personne, un groupe de personnes, un organisme, etc.
 - Le terme *flow* (« flux ») renvoie au passage du produit, du document, de l'information, etc., d'une **étape** à l'autre (*Wikipedia*).
 - ...

L'« industrialisation » de l'informatique



- Méthodologies de développement (fin) :

- « *workflow* » : ...

- proche d'une « *To-Do List* »
(pour le flux),



- ou d'un tableau « *Kanban* »
(pour le qui fait quoi).

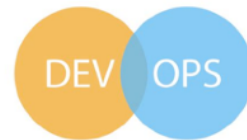


L'« industrialisation » de l'informatique (fin)



- Nombreux contrôles automatisés :
 - dans les langages (*private* contre *public*, etc.)
 - dans les bibliothèques (paramètres vérifiés)
 - dans la programmation
 - gestion de versions (outils Git, SVN, etc.)
 - dans le *workflow*
 - standardisation -> prise en main plus rapide
 - automatisation -> accélération de nombreuses étapes
- effet : sécurité renforcée mais rapidité diminuée

Le DevOps



DevOps



- Définition
 - **Dev** = *développement* logiciel (conception, écriture)
 - **Ops** = administration des infrastructures informatiques (exploitation *opérationnelle*)

DevOps



- Dev = développement logiciel
 - et en particulier :
 - conception,
 - écriture,
 - tests,
 - rendus,
 - adaptation et mise-à-jour

DevOps



- Ops = administration des infrastructures informatiques
 - Parc informatique, Systèmes d'exploitation et Réseaux
 - configuration et paramétrage
 - gestion quotidienne (*'monitoring'*)
 - gestion des pannes
 - mise-à-jour
- ...

DevOps



- Ops = administration des infrastructures informatiques (suite)
 - ...
 - Logiciels
 - installation
 - paramétrage
 - surveillance (*'monitoring'*)
 - mise-à-jour

DevOps



- Dev = développement logiciel (conception, écriture)
- Ops = administration des infrastructures informatiques (exploitation)
- A l'origine, les applications étant de taille limitée et peu intégrées les unes avec les autres.
 - La distinction entre « Dev » et « Ops » était donc quasiment inutile => mêmes personnes pour faire les deux

DevOps (suite)



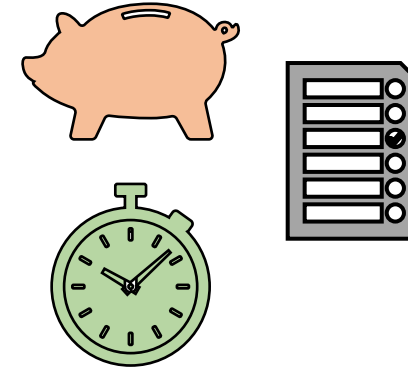
- Mais il y a eu arrivée de nouvelles contraintes :
 - taille grandissante des applications,
 - interdépendances de plus en plus courantes,
 - observation que l'état d'esprit « Ops » est assez différent de l'état d'esprit « Dev » => polarisation des membres de l'équipe, soit plutôt « Ops », soit plutôt « Dev ».
- Il est alors apparu plus efficace de séparer les aspects « Dev » et « Ops » en plaçant les responsabilités respectives dans des équipes séparées.

DevOps (suite)

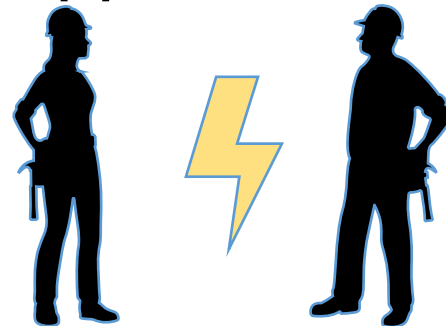


- Note : les trois contraintes de la gestion de projet sont :

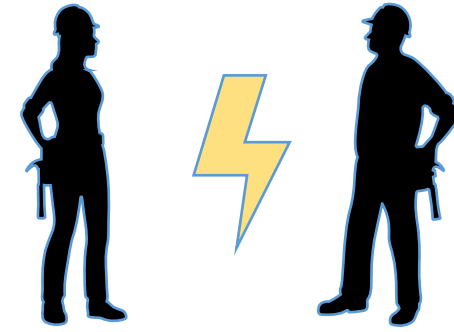
- le coût,
- la qualité/cadre de fonctionnalités,
- et le temps.



- Malheureusement, il y a opposition entre Dev et Ops sur ces contraintes !

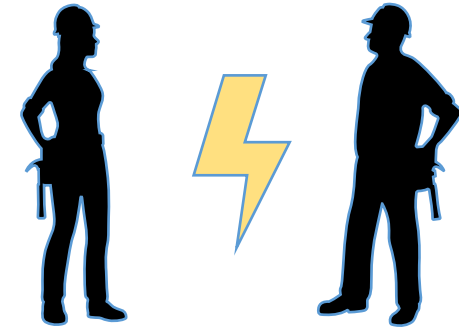


DevOps (suite)



- Pourquoi ?
 - Une équipe **développement** vise à obtenir :
 - des résultats (fonctionnalités) en grand nombre,
 - à moindre coût,
 - et le plus vite possible
 - la qualité et la stabilité des fonctionnalités sont considérés comme moins cruciales et donc *sacrifiables*
 - en particulier en cas de retard ou de dépassement des coûts

DevOps (suite)



- Inversement, une équipe **exploitation** vise à obtenir :
 - des garanties de stabilité dans le maximum de cas,
 - et des garanties de performances sur un système,
- si nécessaire, en *réduisant* ou en *limitant* les fonctionnalités, au moindre doute ou problème, sur le système maintenu
 - en particulier en phase exploitation

DevOps (suite)



- Effet :
 - Cela tend à générer des conflits entre ces équipes en cas de problème !
 - pour les équipes Dev, c'est le matériel ou l'installation (Ops) qui n'est pas au niveau ou qui est défaillant
 - pour les équipes Ops, c'est le logiciel (Dev) qui a des besoins irréalistes
 - => elles vont donc se rejeter les responsabilités les unes sur les autres !

DevOps (suite)



- Réponse actuelle :
 - Le DevOps vise à limiter ces risques de conflits en re-fusionnant les acteurs en une seule équipe.

DevOps (suite)

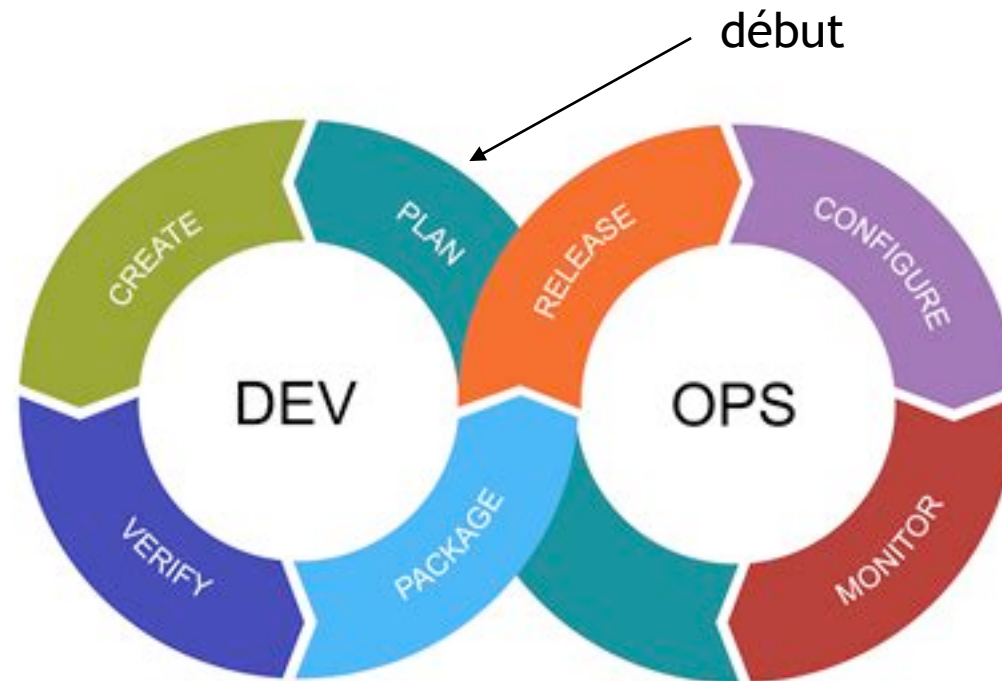


- Moyen :
 - Le DevOps se caractérise principalement par la promotion de l'**automatisation** et du suivi (*'monitoring'*) de toutes les étapes de la création d'un logiciel,
 - depuis le développement, l'intégration, les tests, la livraison,
 - jusqu'au déploiement, l'exploitation et la maintenance des infrastructures,
 - le tout considéré comme une « boucle » qui se répètera

DevOps (suite)



- Une boucle parfois présentée pour DevOps :



DevOps (suite)



- Ceci permet en théorie :
 - des cycles de développement plus courts,
 - une augmentation de la fréquence des déploiements,
 - et des livraisons « continues ».
- Le but est une meilleure atteinte des objectifs économiques de l'entreprise ou du service administratif.
- En pratique, malgré les efforts, la partie « Dev », en amont, a tendance à prendre plus de place que la partie « Ops ».

Le cas du SRE comparé au DevOps



- SRE = '*Site Reliability Engineering*'
 - Ce sont des équipes spécialisées « Ops ».
 - Parfois comparé avec DevOps.
 - Très lié à Google.
- Les équipes de SRE se concentrent sur :
 - La sécurité,
 - Le temps de fonctionnement,
 - Et la capacité à remédier aux problèmes imprévus (fiabilité du site).



Le cas du SRE (suite)

- Note: un pilier important du travail SRE consiste à combattre les incidents
 - les SRE passent donc beaucoup de temps à s'assurer que ces incidents ne se produisent même pas.



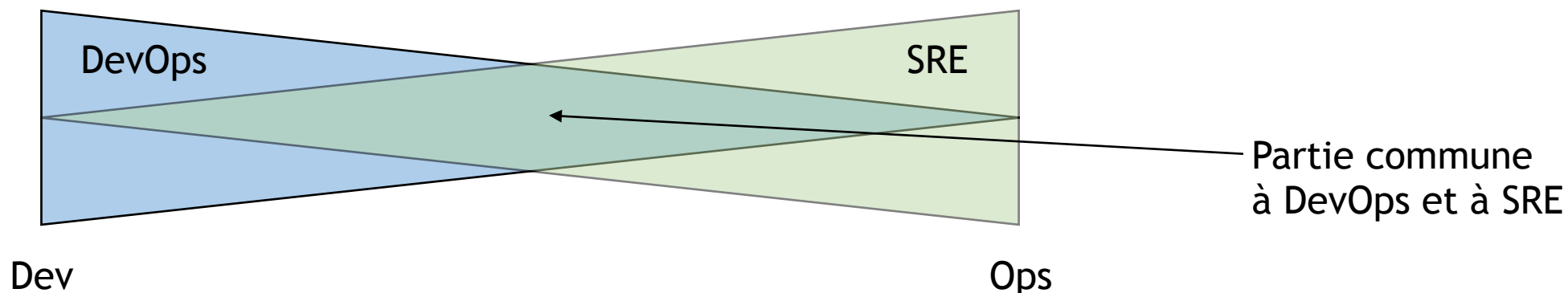
Le cas du SRE (suite)

- En théorie, le SRE retire du travail des équipes Dev certaines des charges complexes liées à la mise à l'échelle et au maintien de la disponibilité des systèmes distribués
 - les pratiques SRE doivent alors permettre aux équipes Dev
 - de se concentrer sur le développement de fonctionnalités
 - plutôt que sur les nuances liées à la réalisation et au maintien des engagements de niveau de service.



Le cas du SRE (fin)

- Mais en fait, les équipes purement SRE ont aussi une partie « Dev »
 - car les choix faits en « Dev » peuvent avoir des effets sur la partie « Ops »
- Donc en pratique, DevOps et SRE ont tendance à se recouvrir !



Le CI/CD



CI/CD



- CI/CD est une automatisation poussée par l'approche DevOps pour gérer toute modification :
 - CI = *Continuous Integration*
 - un « référentiel partagé » reçoit automatiquement toutes les modifications du code source
 - CD = *Continuous Delivery (or Continuous Deployment)*
 - *delivery* : mise automatique de « rendus » dans un dépôt accessible
 - *deployment* : installation automatique directe du dernier « rendu » sur l'environnement de production

CI/CD (suite)



- Fondamentalement, les termes « automatique » et « continu » de CI/CD impliquent une boucle:

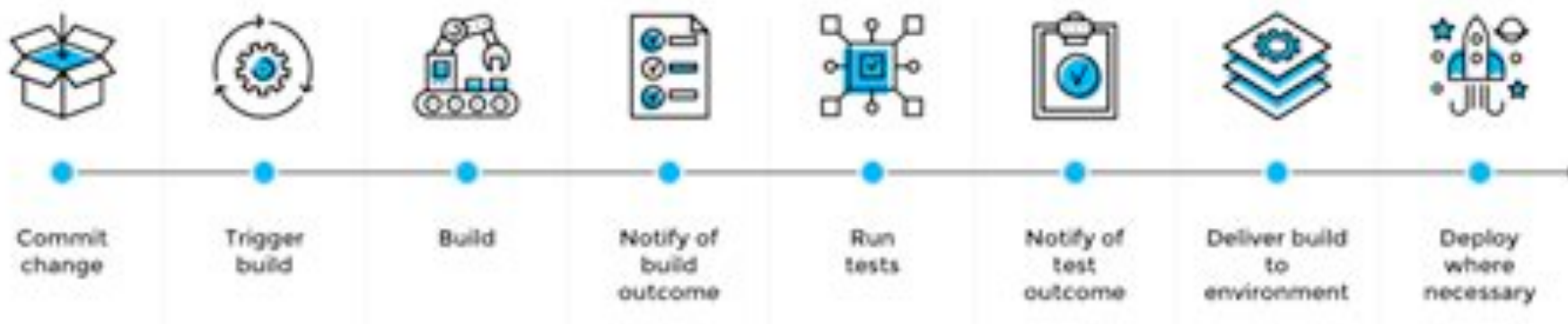
« code changes -> Test -> Build -> Release »
et on recommence...
- L'idée est alors de fractionner les modifications et de les traiter petit morceau par petit morceau, séparément
 - C'est le « pipeline CI/CD » qui va de la modification au déploiement

CI/CD (suite)



- L'idée est de fractionner les modifications et de les traiter automatiquement petit morceau par petit morceau
 - Exemple de « pipeline CI/CD »

CI/CD Pipeline



CI/CD (suite)



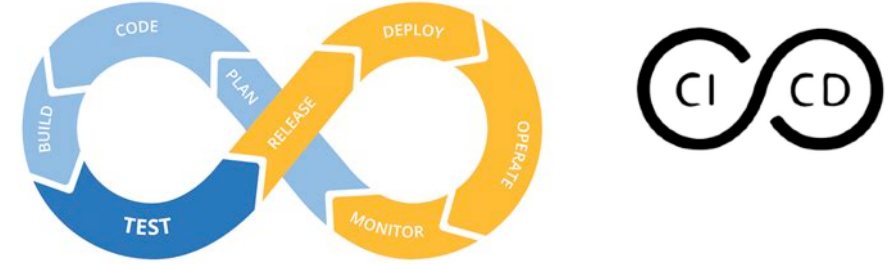
- Remarque: la plupart des outils essentiels au DevOps font partie du processus de CI/CD
 - => Les outils qui servent
 - à l'automatisation de la configuration (comme Ansible, Chef et Puppet),
 - à l'exécution de conteneurs (comme Docker, rkt et cri-o),
 - et à l'orchestration des conteneurs (Kubernetes),
- ne sont pas des outils de CI/CD à proprement parler, mais ils apparaissent dans de nombreux workflows de CI/CD.

CI/CD (suite)



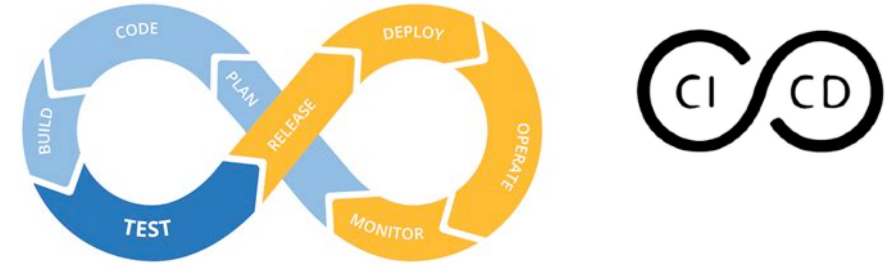
- Exemple de *workflow* proposé pour Git :
 - Dev:
 - 1. Merge code changes
 - 2. put result in remote git repository
 - then CI:
 - 1. get result in git repository
 - 2. make tests
 - 3. build and package
 - 4. put result in artifact repository
 - then CD:
 - 1. get result from artifact repository
 - 2. deploy to step DEV
 - 3. deploy to step STAGING
 - 4. deploy to step PROD

CI/CD (suite)



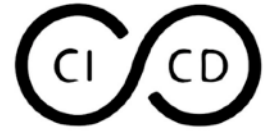
- Quelques recommandations générales de CI/CD :
 - déployer régulièrement les applications : en effet, il est constaté dans la pratique que la répétition contribue à elle seule à mieux fiabiliser le processus !
 - décaler les tests « vers la gauche » : tester au plus tôt améliore aussi la fiabilité
 - pratiquer les tests dans un environnement similaire à celui de production et d'exploitation : limite les « surprises »

CI/CD (suite)

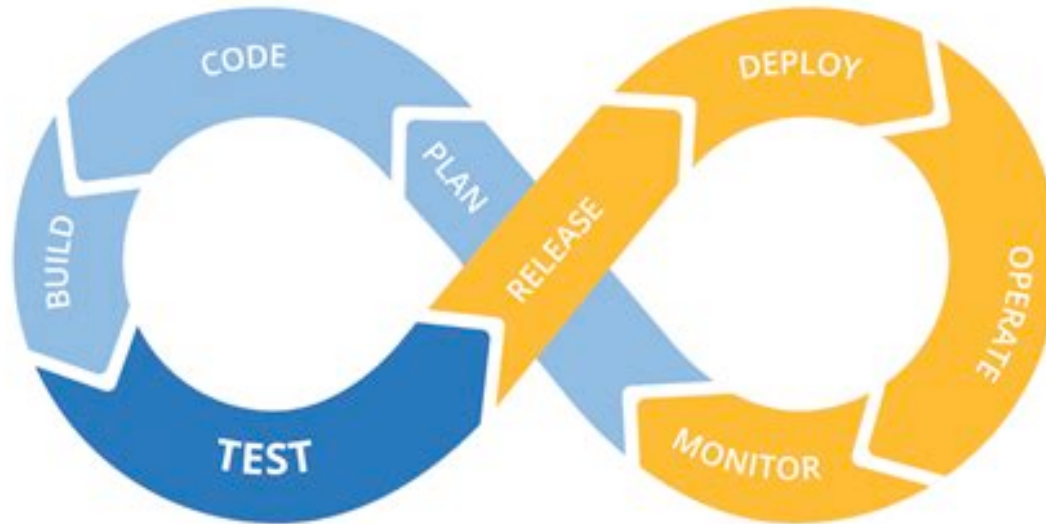


- Quelques recommandations générales (suite) :
 - faire une intégration continue qui inclue des tests continus ;
 - avoir une boucle d'amélioration courte : cela permet un 'feed-back' rapide des utilisateurs) ;
 - surveiller étroitement l'exploitation et la qualité de production, « factualisée » par des métriques et indicateurs clés.

CI/CD (suite)



- Une boucle souvent présentée pour CI/CD (mais aussi DevOps !) :



CI/CD (fin)



- comparaison des boucles (7 étapes contre 8 étapes) :

